LAB4

Lab4

- Implement a **Stack** using both an **array** and a **singly linked list**, and perform **postfix evaluation**.
- If you have any questions, please contact TA through email. (112alsdud@naver.com)
- Lab scores will be uploaded to the LMS next week.
- Questions about Lab 4 scores will only be accepted during next lab session.

- Code name: {student_id}_p4_array.c, {student_id}_p4_linked_list.c
- **Submission deadline:**
  - {student_id}_p4_array.c: **During class time (3/26, 1:00 PM)**
  - {student_id}_p4_linked_list.c: **By 3/26, 11:59 PM**
- The code will be tested with 5 different input files.
- Each input file will be worth 20 points
- Make sure that the output format matches the provided example exactly.

Evaluation criteria

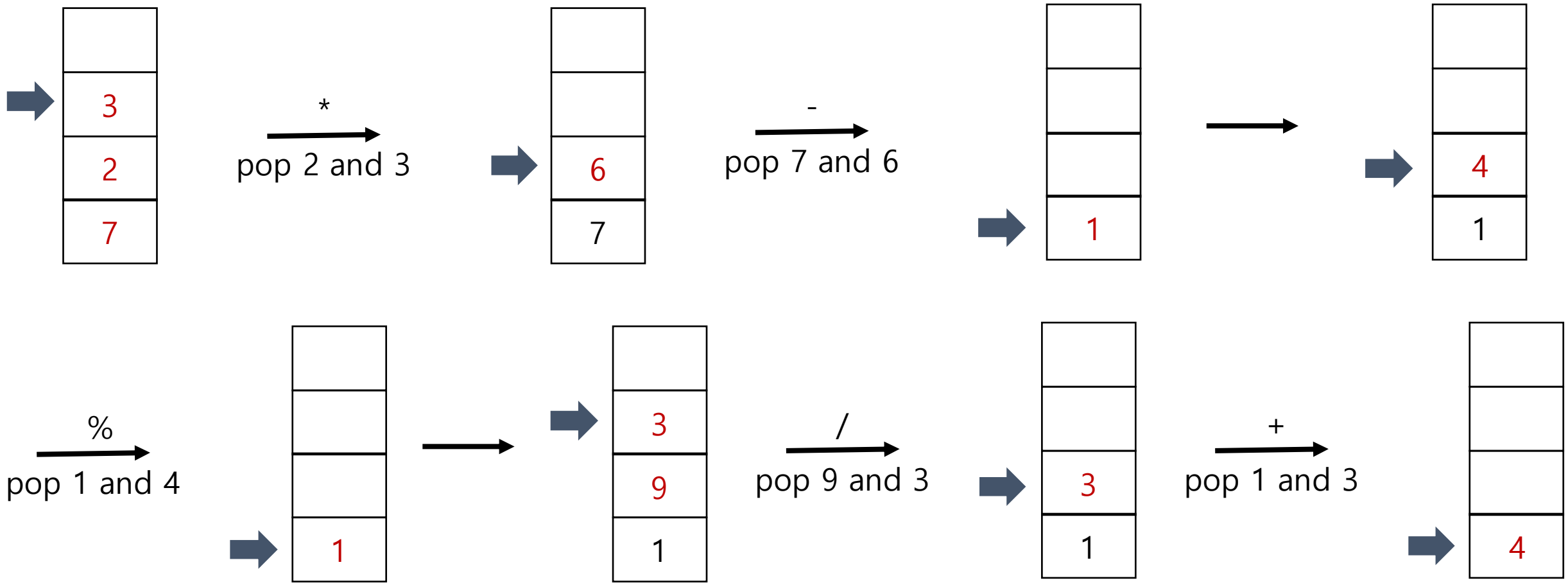
Category	Evaluation	
p4_array	100	
p4_linked_list	100	
Total	200	

- *Use GCC **13.3** version.*
- *No score will be given if there is a compile error caused by difference of gcc version*

Lab 4. Postfix evaluation using Stack

- **Given postfix : 7 2 3 * - 4 % 9 3 / +**

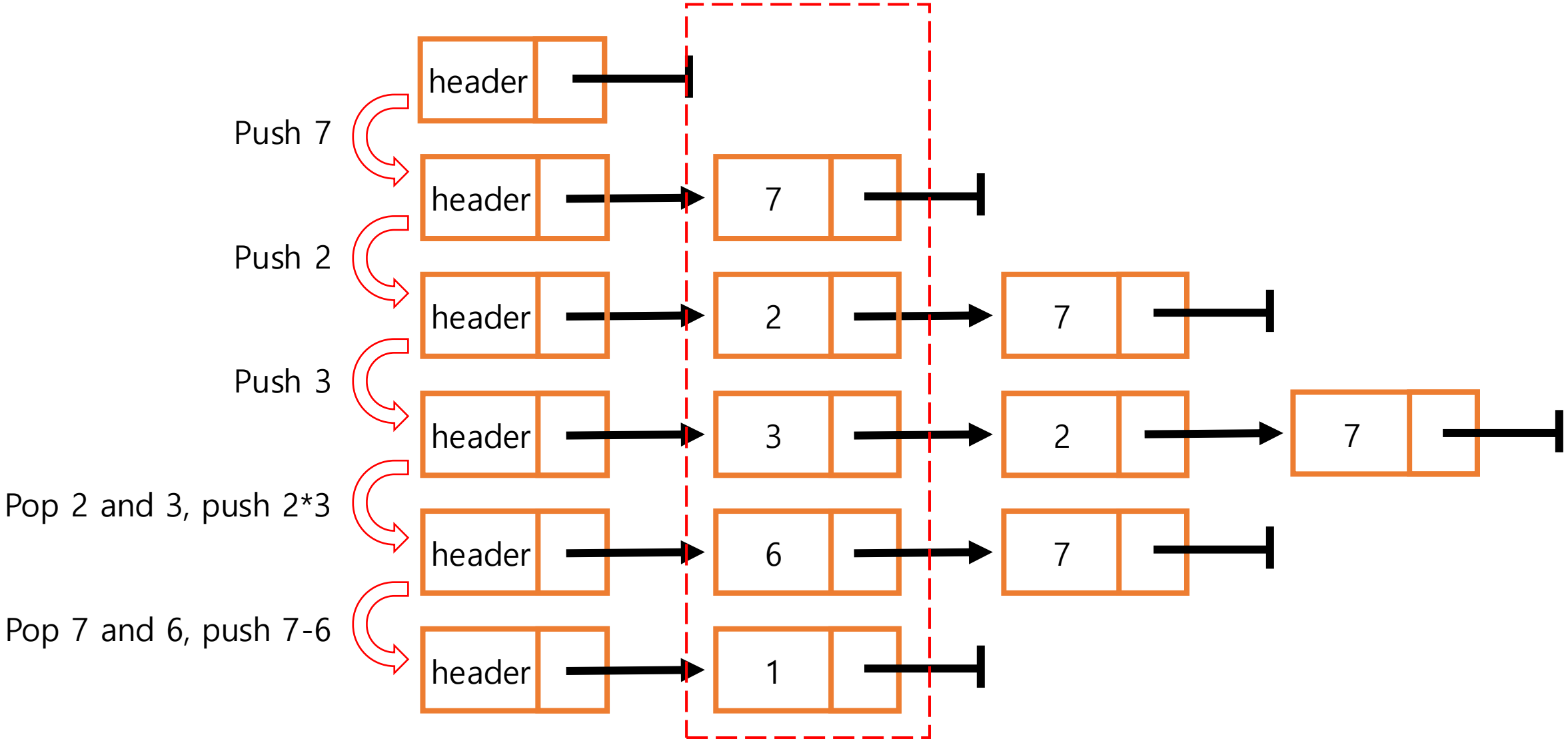
➡ : Top of stack



Lab 4. Postfix evaluation using Stack

: Top of stack

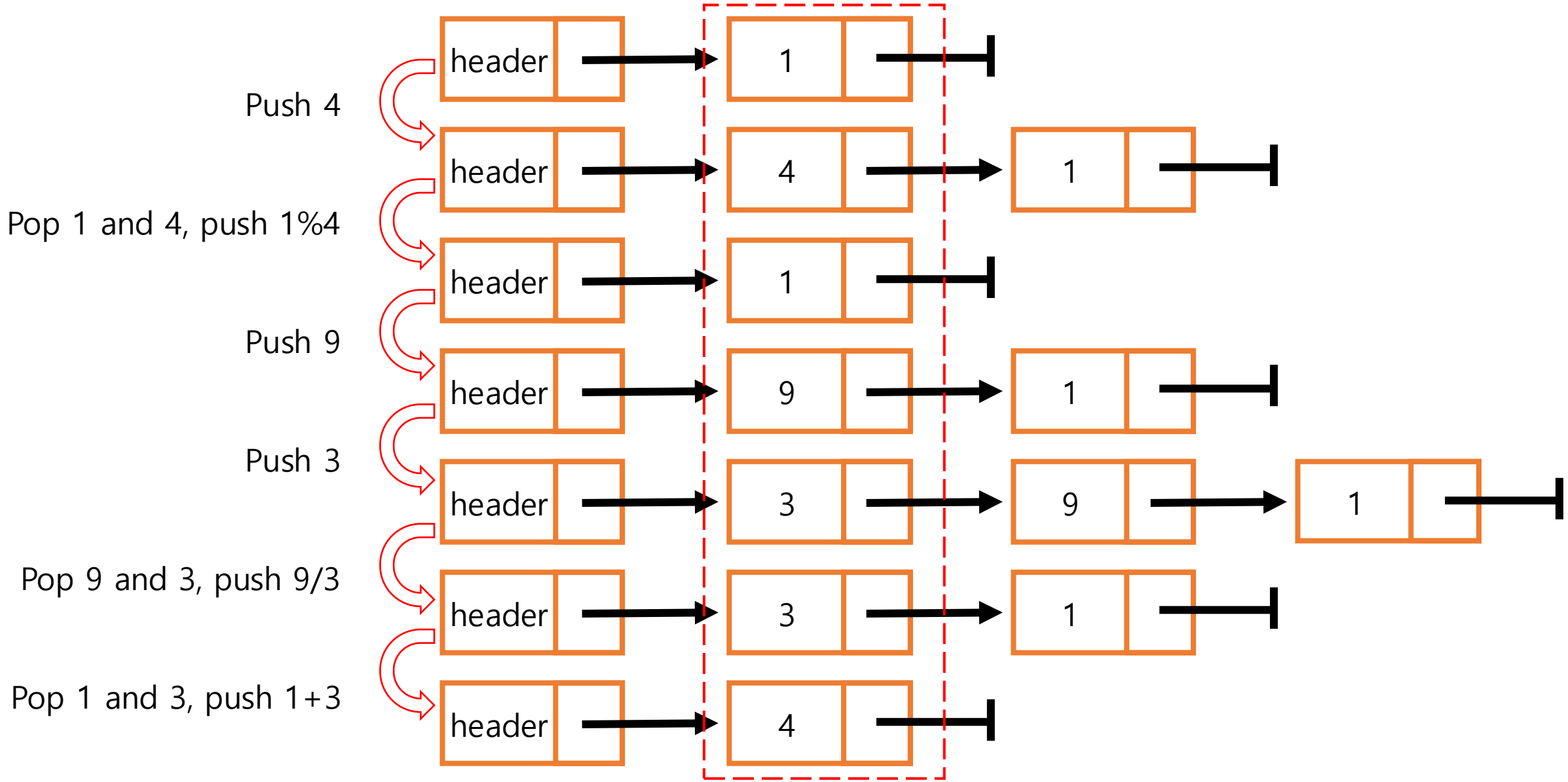
- **Given postfix : 7 2 3 * - 4 % 9 3 / +**



Lab 4. Postfix evaluation using Stack

: Top of stack

- **Given postfix : 7 2 3 * - 4 % 9 3 / +**



Lab 4. Postfix evaluation using Stack

- Available operators: +, -, *, /, and %
- Operands: single-digit numbers (1, 2, 3, 4, 5, 6, 7, 8, and 9)
- Conditions:
  - The stack must be implemented using both an **array** and a **singly linked list.**
  - The expression should be no more than 100 characters.
  - The delimiter for the end of the expression is '**#**'.
  - Max stack size is **10**.
- There are two rules for popping and pushing the operands from/to the stack:
  - When you meet an operand (number), push it into the stack.
  - When you meet an operator, pop two operands from the stack, perform the operation, and push the result back to the stack.
  - **Every time there is a push, print out the entire stack.**

Lab 4. Postfix evaluation using Stack

- **Your program should include:**
  - Stack CreateStack();
  - void MakeEmpty(Stack S);
  - void PrintStack(Stack S);
  - int IsEmpty(Stack S);
  - int IsFull(Stack S);
  - void Push(ElementType X, Stack S);
  - int Pop(Stack S);
  - void DeleteStack(Stack S);
  - void Postfix(Stack S, char c);

Lab 4. Postfix evaluation using Stack

- **Stack CreateStack()**
  - Create an empty stack with the size of max → for array implementation.
  - Create an empty stack with a header node → for linked list implementation.
- **void PrintStack(Stack S)**
  - Print all elements in the stack from top to bottom
- **int IsEmpty(Stack S)**
  - Check whether the stack is empty or not.
- **int IsFull(Stack S)**
  - Check whether the stack is full or not.
- **void Push(ElementType X, Stack S)**
  - Push a new element X in the stack.
  - If your stack is full, just print an error message **"Stack Overflow!"** and exit the program.

Lab 4. Postfix evaluation using Stack

- **int Pop(Stack S)**
  - Pop the top element from the stack and return it.
  - If the stack is empty, just print an error message **"Stack Underflow!"** and exit the program.
- **void MakeEmpty(Stack S)**
  - Remove all elements from the stack
- **void DeleteStack(Stack S)**
  - Delete the entire stack and free all the memory allocated to stack.
  - Use with MakeEmpty function.
- **void Postfix(Stack S, char c)**
  - When you meet an operand (number), push it into the stack.
  - When you meet an operator, pop two operands from the stack, perform the operation, and push the result back to the stack.
  - If you meet an unavailable operator, print an error message **"Invalid operator encountered: {operator}"** and exit the program.

Lab 4. Postfix evaluation using Stack

- Structure

```
typedef struct StackRecord {
    ElementType *key;
    int top;
    int max_stack_size;
} StackRecord;


typedef struct Node {
    ElementType Element;
    PtrToNode Next;
} Node;
```

- Function

```
Stack CreateStack();
void MakeEmpty(Stack S);
void PrintStack(Stack S);
int IsEmpty(Stack S);
int IsFull(Stack S);
void Push(ElementType X, Stack S);
int Pop(Stack S);
void DeleteStack(Stack S);
void Postfix(Stack S, char c);
```

Lab 4. Example

Ex 1)

```
~/Dropbox/연구실/자료구조/2026자료구조/실습/Lab4/code ❯ cat input_ex1.txt
723*-4%93/+#
~/Dropbox/연구실/자료구조/2026자료구조/실습/Lab4/code ❯ ./p4 input_ex1.txt
Current Stack: 7
Current Stack: 2 7
Current Stack: 3 2 7
Current Stack: 6 7
Current Stack: 1
Current Stack: 4 1
Current Stack: 1
Current Stack: 9 1
Current Stack: 3 9 1
Current Stack: 3 1
Current Stack: 4

Calculating Done!
Evaluation result: 4
```

Ex 2)

```
~/Dropbox/연구실/자료구조/2026자료구조/실습/Lab4/code ❯ cat input_ex2.txt
123456789362#
~/Dropbox/연구실/자료구조/2026자료구조/실습/Lab4/code ❯ ./p4 input_ex2.txt
Current Stack: 1
Current Stack: 2 1
Current Stack: 3 2 1
Current Stack: 4 3 2 1
Current Stack: 5 4 3 2 1
Current Stack: 6 5 4 3 2 1
Current Stack: 7 6 5 4 3 2 1
Current Stack: 8 7 6 5 4 3 2 1
Current Stack: 9 8 7 6 5 4 3 2 1
Current Stack: 3 9 8 7 6 5 4 3 2 1

Stack Overflow!
```

Ex 3)

```
~/Dropbox/연구실/자료구조/2026자료구조/실습/Lab4/code ❯ cat input_ex3.txt
1942^-+#
~/Dropbox/연구실/자료구조/2026자료구조/실습/Lab4/code ❯ ./p4 input_ex3.txt
Current Stack: 1
Current Stack: 9 1
Current Stack: 4 9 1
Current Stack: 2 4 9 1
Invalid operator encountered: ^
```

Ex 4)

```
~/Dropbox/연구실/자료구조/2026자료구조/실습/Lab4/code ❯ cat input_ex4.txt
285/+%#
~/Dropbox/연구실/자료구조/2026자료구조/실습/Lab4/code ❯ ./p4 input_ex4.txt
Current Stack: 2
Current Stack: 8 2
Current Stack: 5 8 2
Current Stack: 1 2
Current Stack: 3

Stack Underflow!
```

Lab 4. Sample code

```c
#include <stdio.h>
#include <stdlib.h>
#include <ctype.h>

#define MAX_STACK_SIZE 10

typedef int ElementType;

typedef struct StackRecord {
    ElementType *key;
    int top;
    int max_stack_size;
} StackRecord;

typedef StackRecord *Stack;

Stack CreateStack(void);
void MakeEmpty(Stack S);
void PrintStack(Stack S);
int IsEmpty(Stack S);
int IsFull(Stack S);
void Push(ElementType X, Stack S);
int Pop(Stack S);
void DeleteStack(Stack S);
void Postfix(Stack S, char c);
```

```c
int main(int argc, char* argv[]) {
    if (argc != 2) {
        printf("Please enter an input file!");
        exit(1);
    }

    FILE* fi = fopen(argv[1], "r");

    if (fi == NULL) {
        perror("Error opening file");
        exit(1);
    }

    Stack stack = CreateStack();

    char c;
    while (fscanf(fi, "%c", &c) != EOF) {
        if (c == '#') {
                printf("\nCalculating Done!");
                break;
        }
        Postfix(stack, c);
    }
    printf("\n");
    printf("Evaluation result: %d\n", Pop(stack));

    fclose(fi);
    DeleteStack(stack);
}
```